

业务全景与端到端主线

gomall 一次购物的端到端全景——从进门到收货怎么跑通 / 卡在哪

gomall · 业务侧 deck

业务全景：gomall 到底在做什么

系统架构：分层拓扑与领域地图

端到端主线：一次成功购物的”黄金路径”

贯穿全局的硬骨头：超卖·一致·限流·降级·可观测

订单状态机：把 7 跳串成一条可查的线

业务全景：gomall 到底在做什么

gomall 是什么：一个讲清“为什么这么选”的电商后端

用 **Go + Gin** 写的电商后端，把一笔订单从浏览 → 搜索 → 下单 → 支付 → 履约的全链路跑通；再叠上优惠券 / 秒杀 / 红包 / 拼团 / 预售、Web3 支付、ES+Milvus 向量检索。

它到底在解决什么

- **解决真实问题**：盯住真实电商里“该怎么选”的工程决策——超卖、一致性、限流、降级如何权衡。
- **20 个业务域**按 DDD 垂直切片，每域 internal/< 域 >/ 五件套内聚，跨域写经属主服务落库。
- **4 条支付通道**（钱包 / Stripe / USDC / ETH）殊途同归，统一汇到复式记账台账，借贷守恒可对账。

技术栈一览

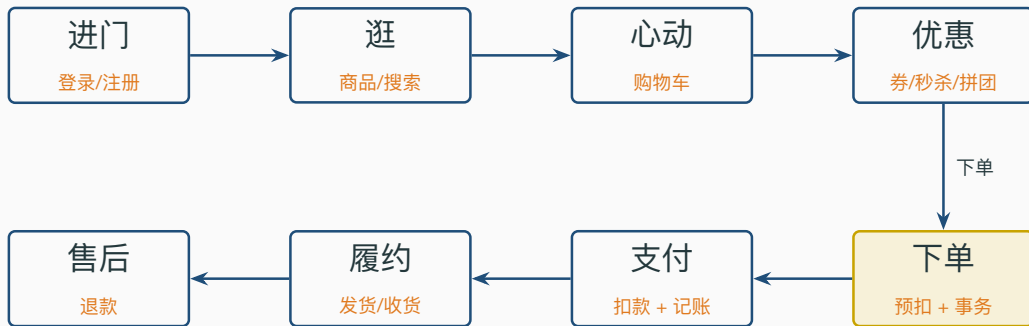
- **接入**：Gin + 中间件链（限流 / JWT / RBAC / 熔断 / 幂等 / Jaeger）
- **存储**：MySQL 主从 · Redis · Elasticsearch · Milvus
- **异步**：RabbitMQ + Outbox 旁路投递
- **链上**：Web3 Escrow + EVM 对账
- **可观测**：Jaeger 链路 + SkyWalking

先认人：一笔订单背后站着 5 个角色

- **C 端用户**：进门 → 逛 → 下单 → 收货。只关心快、对、不丢。
- **商家 (Boss)**：订单是”我能不能发货”的唯一源头；货卖出去钱要到账。
- **运营**：办活动拉量——优惠券 / 满减 / 秒杀 / 拼团 / 预售，预算不能被刷穿。
- **客服**：用户来问”我的钱去哪了 / 货到哪了”，得有一条能查的状态线。
- **SRE**：大促洪峰别把库存卖超、别把数据库打挂，挂了要能降级。

这份概览把一次购物从**进门到收货**串成一条线：先看全局怎么连，再逐段看每一跳背后的技术取舍，以及”出事会卡在哪”。

业务全景图：一次购物穿过的 8 个环节



贯穿全局：库存防超卖·Outbox 一致性·流量治理·优雅降级

哪里会出事：把”坑”标在主线上（只盯 happy path 时全看不见）

这一跳	容易翻车的点（学生常以为没事）	后果	本 deck 怎么填
逛 / 详情	热点缓存集体失效、回源时击穿/穿透/雪崩	DB 打挂	Cache Aside+ 空值缓存
优惠 / 秒杀	券被脚本超领、预算被刷穿、库存卖超	营销亏钱	两桶库存 Lua+ 预算降级
下单	(1) 超卖 (2) 双写丢消息 (3) 重试重复下单	钱货两失	防超卖 + Outbox+ 幂等
支付	第三方重复回调、扣了钱没货、台账对不平	直接资损	同事务收钱 + 复式记账
履约 / 售后	并发改乱订单状态、同一笔退款退两次	状态错乱	状态机 CanTransition
大促全局	零点洪峰打满连接池、下游挂了拖死自己	整站雪崩	限流 + 熔断 + 削峰

凡是碰钱、碰库存的地方都会翻车——越靠近”下单一支付”代价越高，事务 / 原子性 / 资源都堆这段；边缘能力（搜索 / 优惠 / 通知）一律可降级，绝不反过来阻断 happy path。

逐个拆解见”贯穿全局的硬骨头：超卖·一致·限流·降级”与”订单状态机”两节

业务承诺：我们对外兜什么底（SLO）

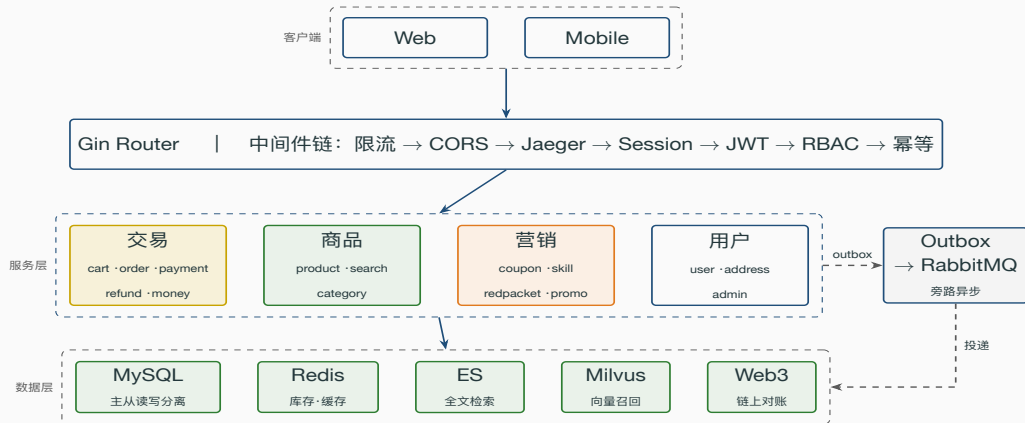
一个交易系统的可信度，不在功能多，而在敢把哪几条线写进合同——出事要赔的那种。gomall 把承诺分成 P0~P3 四档，资源向核心交易倾斜。

等级	典型场景	可用率	p99 延迟
P0 核心交易	下单 / 支付	99.95%	< 500ms
P1 核心读	商品详情 / 订单列表	99.9%	< 200ms
P2 营销秒杀	抢券 / 秒杀 / 红包	99.9%	< 200ms
P3 信息类	轮播 / 分类	99.5%	< 1s

- 钱货不丢：支付与库存、流水落同一事务，崩了整笔回滚，绝不“扣了钱没货”。
- **happy path** 不被周边阻断：优惠 / 搜索 / 通知挂了，下单支付照走。
- 不能上游 **429**：P0 靠缓存 + 削峰扛峰值，宁可下游慢。
- 阶梯故意压：每加 1 个 9 成本翻番；限流返业务码不算宕机，故 P2 也 99.9%。

系统架构：分层拓扑与领域地图

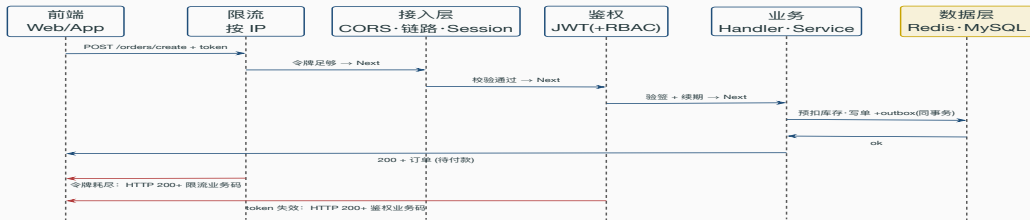
gomall 整体架构：一条请求从客户端落到数据层



核心链路同步落库收钱，事件经 Outbox 旁路异步散给搜索 / 统计 / 履约，主线不被下游拖慢。

中间件洋葱模型：下单请求逐层穿过鉴权打到后端（真实顺序）

中间件是洋葱模型：逐层 `c.Next()` 进内层、再原路返回出；任一层 `Abort` 就在那层折回，到不了业务。

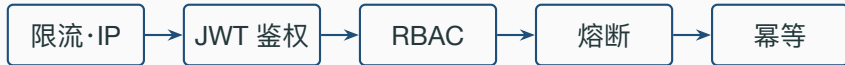


真实顺序：限流 (IP) → CORS → Jaeger → Session → JWT；admin 再叠 RequireRole、下单末端挂 Idempotency。限流在最外层、洪水在最便宜处被拒；按用户的秒杀滑窗限流在鉴权之后。

`routes/router.go:41-55` 全局链；`order/routes.go:12` 下单叠 `Idempotency`；被拒一律 `HTTP 200+` 业务码

中间件链：顺序即设计（为什么是这个次序）

顺序不是随手排的——每一环只在前一环放行后才有意义，错一步就是漏洞或浪费。下面按 `router.go` 真实注册顺序逐环讲。



- **限流·IP** `TokenBucket(100,200)`：放最外层——匿名洪水在最便宜处被拒，不浪费后面的解析；被拒 HTTP 200 + 限流业务码。
- **JWT 鉴权** `AuthMiddleware()`：解 token 拿 `user_id`，挂 `authed` 组；失败 HTTP 200 + 鉴权码（项目统一用 `body status`，不用 401）。
- **RBAC** `RequireRole("admin")`：登录之上叠角色校验，只护 `admin` 组——必须先认证再鉴权。
- **熔断**：支付等路由对下游调用加熔断，连错 5 次 open 10s，快速失败不被垂死依赖拖垮。
- **幂等** `Idempotency()`：放最后——确认要执行业务时才用 Redis 锁去重，避免给被拒请求白占幂等键。
- **注**：按用户维度的秒杀滑窗限流在鉴权之后（要先知道是谁），全局按 IP 的令牌桶才放最前。

数据层选型与边界：一个 MySQL 为什么不够

把全部活儿压给一个 MySQL，要么慢死、要么挂掉。正解按”访问形态”分流：强一致归 MySQL，高并发与检索归旁路，旁路挂了能退回 DB。

存储	扛什么	挂了怎么办
MySQL 主从	交易事务、权威读	核心依赖，不可降级
Redis	库存预扣 / 热点缓存 / 限流	缓存未命中回源 DB
Elasticsearch	商品全文检索	退回 DB LIKE 查询
Milvus	语义向量召回	退回纯 ES 关键词召回

- **读写分离**：dbresolver 写走 Sources、读走 2 个 Replicas，RandomPolicy 负载均衡。
- **搜索降级**：ES 不可用时落到 DB name/info LIKE；语义召回报错则退回纯向量——周边能力只增强体验，绝不阻断 happy path。

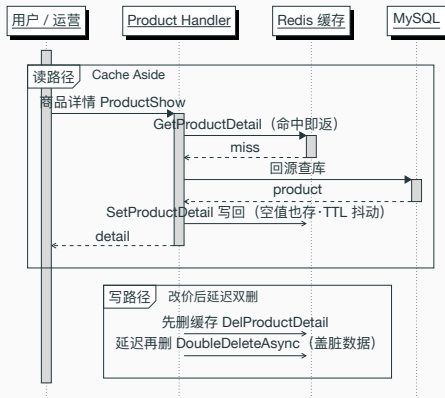
端到端主线：一次成功购物的”黄金路径”

黄金路径：7 跳，每一跳都可能掉链子

1. 登录——拿到 JWT，后续每个请求带着它过鉴权中间件。
2. 看商品——详情走 Cache Aside，热点不打穿 DB。
3. 加购 / 选地址——没地址下不了单，默认地址兜底。
4. 下单——算优惠 → 预扣库存 → 一个事务落订单 + 事件。
5. 支付——扣余额 + 改状态 + 真正消库存，同事务发”已支付”事件。
6. 发货 / 收货——状态机推进，每一步只允许合法迁移。
7. 结清——订单 Completed；中途不付钱则 15 分钟自动关单。

这条线横跨 user / product / cart / promo / order / inventory / payment 七个包，
任何一跳的失败都不能让用户”钱货两失”——这是整套设计的第一性原则。

看商品：Cache Aside 读路径 + 更新时延迟双删

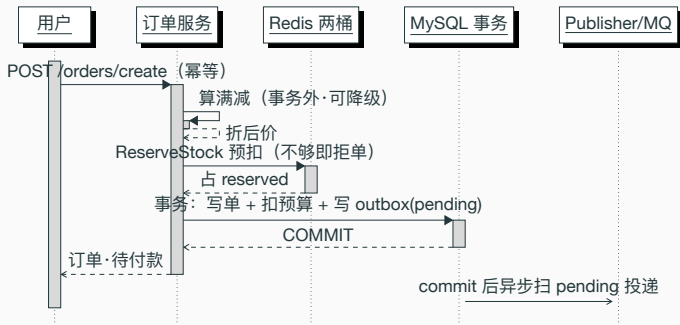


internal/product/service.go:41 ProductShow (回源 + 写回 + 更新双删); repository/cache/product.go

GetProductDetail / SetProductDetail / DoubleDeleteAsync

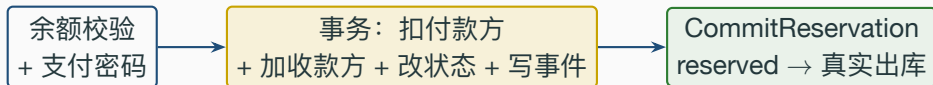
端到端时序：下单这一跳的内部（先预扣、再事务、后通知）

三段式：事务外算满减/预扣库存（可降级）→ 一个事务写单 + 扣预算 + outbox
→ 事务后异步投 MQ；失败则 ReleaseReservation 退预扣。



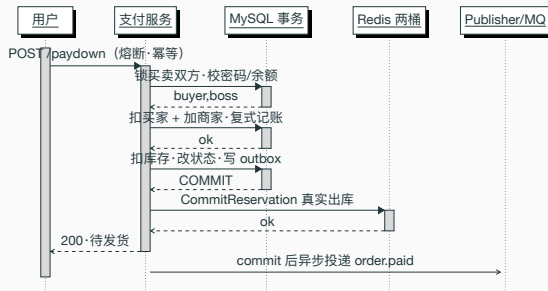
支付这一跳：钱和库存在这里才真正落定

- 下单时库存只是 **reserved** (预占)，钱还没动——订单状态 WaitPay。
- 支付成功才在一个事务里：扣用户余额 → 加商家余额 → 改状态 WaitShip → 写 order.paid 事件。
- 事务提交后同步 CommitReservation：把 reserved 真正消掉，库存数字才最终减少。



internal/payment/service.go:36 PayDown; repository/cache/inventory.go:98 CommitReservation

端到端时序：支付这一跳的内部（钱·库存·状态·事件一个事务落定）



下单只预占库存、不动钱；支付才在一个事务里把扣买家 + 加商家 + 复式记账 + 扣库存 + 改状态 + 写 order.paid 一并落定，全成才 COMMIT，任一守卫失败整笔回滚——绝不”扣了钱没货”。

internal/payment/service.go:55 Tx :87 锁双方:123/:150 复式记账:162 扣库存:173 改状态:205 outbox :224
CommitReservation

贯穿全局的硬骨头：超卖·一致·限流·
降级·可观测

硬骨头 1：库存防超卖——为什么不用 UPDATE WHERE stock>0

- 高并发下行锁排队 + 尾延迟爆炸；下单链路最怕在事务里等锁。
- gomall 把库存搬到 Redis，用 **两个桶**：available（可卖）/ reserved（已占）。
- 预扣 / 提交 / 释放都是 **单个 Lua 脚本原子完成**，一次 RTT，天然不超卖。

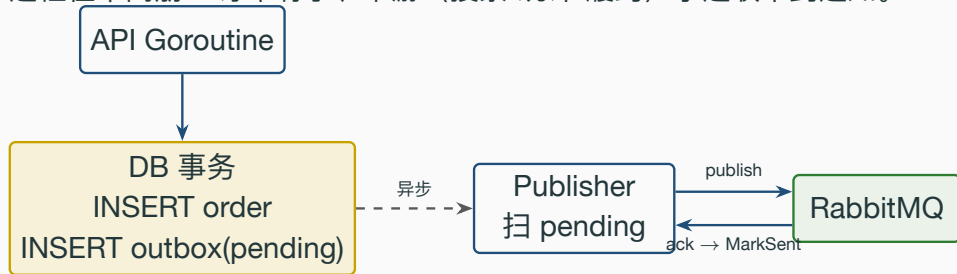
Listing 1: reserveScript: available 与 reserved 两个 key 原子流转

```
1 -- KEYS[1]=available 桶 KEYS[2]=reserved 桶 ARGV[1]=n
2 local avail = redis.call('GET', KEYS[1])
3 if avail == false then return -2 end -- 库存未初始化
4 if tonumber(avail) < tonumber(ARGV[1]) then
5     return -1 -- 不够，拒单
6 end
7 redis.call('DECRBY', KEYS[1], ARGV[1]) -- available -= n
8 redis.call('INCRBY', KEYS[2], ARGV[1]) -- reserved += n
9 return 1
```

repository/cache/inventory.go:32 reserveScript ·:78 ReserveStock 封装·:98/:113 Commit/Release

硬骨头 2: Outbox ——解决” 订单存了但消息丢了”

双写问题：先 INSERT order 再 publish MQ，两个动作不在一个事务里。
进程在中间崩 = 订单有了、下游（搜索/统计/履约）永远收不到通知。



订单与事件**同生共死**写进一个事务；投递交给独立 publisher 重试。
至少一次投递 + 下游幂等消费 = 事件不丢。

硬骨头 3：优雅降级——核心稳如磐石，边缘随时可弃

能力	挂了会怎样	gomall 的选择
满减计算	算不出折扣	降级为 无折扣 ，照样下单
满减预算	预算抢光	事务内 改写为无折扣 ，不报错
RabbitMQ	延迟关单不可用	DB Cron 定时扫描 兜底关单
ES / Milvus	搜索退化	退回 DB LIKE 路径

判断标准只有一句：它是不是“用户付钱拿货”的必经环节？

是 → 进事务、保原子；不是 → 失败降级，绝不阻断 happy path。

internal/order/service.go:78 满减降级 ·:126 预算耗尽降级；cmd/main.go:69-102 ES/Milvus 静默跳过

业务痛点：用户手抖狂点”提交订单”、客户端超时自动重试、弱网下 App 补发请求——同一笔下单不能变成三笔。

请求态状态机 (Redis Hash)

- init: 已发券、未执行
- processing: 首单执行中, 后到一律挡回
- done: 已完成, 缓存响应直接回放

一段 Lua 单次 RTT 原子完成”读态 + 抢锁”。

Listing 2: acquire: init 到 processing 原子推进

```
1 local v = redis.call('HGET', KEYS[1], 'state')
2 if v == false then return {0, ''} end
3 if v == 'done' then return {2,
4     redis.call('HGET', KEYS[1], 'result')} end
5 if v == 'processing' then return {3, ''} end
6 -- init: 抢到执行权
7 redis.call('HSET', KEYS[1], 'state', 'processing')
8 redis.call('EXPIRE', KEYS[1], tonumber(ARGV[1]))
9 return {1, ''}
```

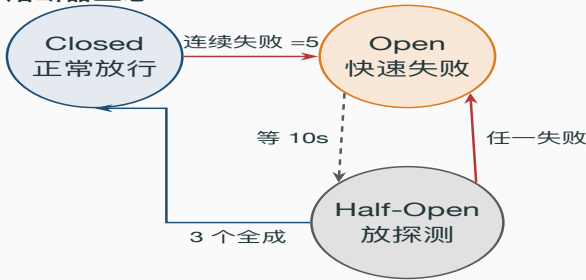
压测实证：同 Idempotency-Key、50 VU、15s 累计 **755,033** 次请求 → DB 中

两道闸门：令牌桶在入口按 IP 限速，挡住脚本刷接口；熔断器在出口监控下游，下游连续报错就快速失败，不让请求堆在垂死的依赖上拖垮自己。

令牌桶限流（每 IP）

- 全局 TokenBucket(100, 200)：每 IP 每秒补 100 个令牌，桶容量 200 (允许突发)。
- 取不到令牌 → 直接 Abort，返回”请求过于频繁”。
- 后台 janitor 周期回收 10 分钟无访问的 IP 条目，map 有界，防海量 IP 撑爆内存。

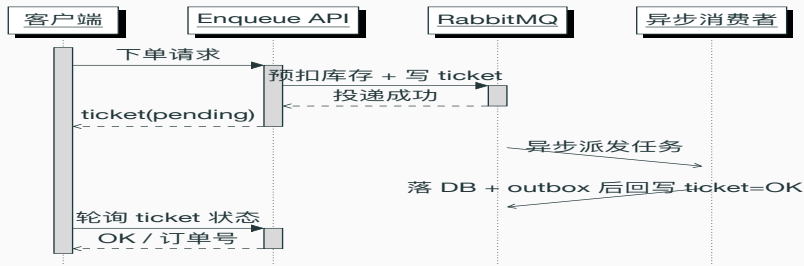
熔断器三态



细节：半开探测带”代际”令牌，上一轮 Open 期的迟到回报会因代际不符被

削峰：下单先拿 ticket，洪峰摊到 MQ

业务背景：大促零点瞬时洪峰，若每个请求都同步写库，DB 连接池秒满。改成“先预扣库存 + 投 MQ 拿 ticket 秒回”，后台慢慢消费落单，把尖峰摊平成平稳的消费速率。



ticket = 雪花 ID 存 Redis 带 TTL；写 ticket / marshal / publish 三个失败点任一出错都先 ReleaseReservation 退回预扣库存，绝不让库存空占。

可观测性：一次下单串成一条 trace

下单跨了网关、订单、库存、MQ、DB 多个环节。出问题时光看单机日志拼不出全貌——链路追踪给每次请求发一个 `traceId`，沿调用一路透传，把散落的 `span` 串成一条完整时间线。

链路怎么串起来

- **两套并存**：`cmd/main.go` 匿名引入 `skywalking-go agent`（进程级旁路自动埋点，无业务侵入）+ `track.go` 的 `Jaeger()` 中间件（OpenTracing）。
- 真正的跨服务透传走 **Jaeger()**：读请求头 `uber-trace-id` 续上父 `span`；缺失或解析失败则降级开本地 `span`，绝不吞掉整条请求。
- `span` 放进 `context` 往下传，DB/Redis/MQ 调用挂成子 `span`。

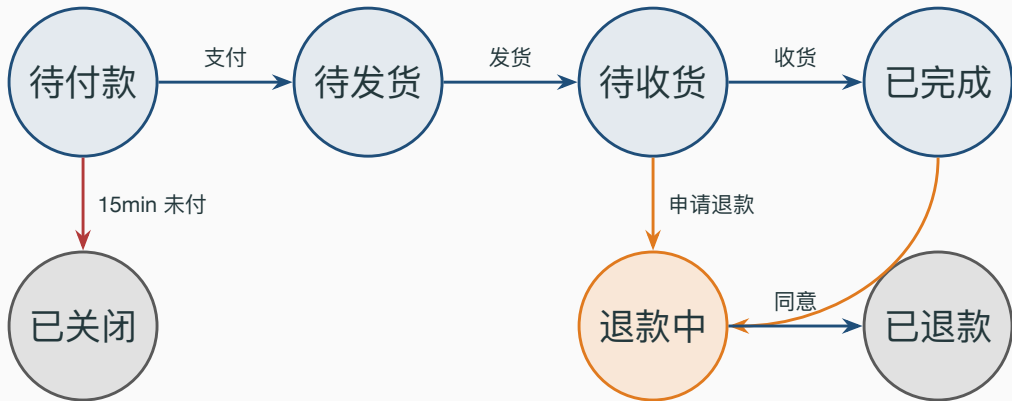
关键指标看板

指标	用途
QPS	吞吐 / 容量水位
p95 延迟	尾延迟，体感真相
错误率	非 2xx 占比

业务态（限流/熔断/幂等命中）HTTP 仍 200，单独用 `status` 码区分，避免污染错误率口径。

订单状态机：把 7 跳串成一条可查的
线

订单不是一个布尔值，是一台状态机



每一次状态变更都走 CanTransition 校验，非法迁移直接报错——
“已完成”的订单不可能跳回“待付款”，客服查到的状态永远自治。

这套架构的一句话总结

核心链路（扣库存 + 落单 + 收钱）必须稳如磐石，周边能力（优惠 / 通知 / 搜索 / 关单）全部可降级。

顺序即设计：

- 能在事务外做的绝不进事务——满减、库存预扣。
- 进了事务就保证原子——订单 + 优惠预算 + outbox 事件。
- 事务外的副作用必须能补偿——库存释放、延迟关单。

internal/order/service.go:63-178 OrderCreate —函数浓缩全部原则